immutables: Persistent, Monoid-Annotated Finger Trees for R

Shawn T. O'Neil
TISLab

Abstract

We present **immutables**, an R package providing persistent finger-tree-backed collections (catenable sequences, priority queues, ordered sequences, and interval indices) with user-extensible monoid annotations. The same predicate-based locate and split primitives power every built-in operation and remain accessible to users, enabling $O(\log n)$ queries over custom aggregates. We describe the API, provide micro-benchmarks against base R and existing packages, and demonstrate the extension points with a DNA prefix-search example.

Keywords: R, finger trees, persistent data structures, monoids.

1. Introduction

The R programming language is a natural first choice for statistical analyses, but its ever-growing popularity has diversified its application to domains as broad as web services (TOD p), user interfaces (TOD r), machine learning (TOD n), simulation (TOD o), mobile computing (TOD t), computational geometry (TOD m), graph algorithms (TOD e), and much more (TOD h). Data structures, or collections, are foundational building blocks for almost all computing, providing the features necessary for efficient in-memory data storage and retrieval.

Ordinal-indexed arrays, matrices, and lists are ubiquitous in statistical applications; double-ended queues support fast insertion and removal from ends and are widely used in state-search and scheduling applications (TOD c); key-ordered sequences provide fast search, access, and range queries (TOD i); priority queues associate elements with priority values for efficient access by minimum or maximum priority, often used in graph algorithms and simulation (TOD l). Amongst more exotic structures, pretty trees find application in bioinformatics (TOD k), and interval trees are widely used in computational geometry (TOD f).

Data structure implementations generally fall in one of two classes: mutable or immutable (also known as persistent). Operations on mutable structures allow the internal structure or contents to change. Operations on immutable structures, by contrast, return changed copies, preserving the original for further use. Purely functional languages like Haskell and ML endorse persistence, and while mutating data structures are generally easier to implement and faster, the functional programming community has invested significant effort in designing fast immutable implementations for a wide variety of applications (TOD x). This immutability is familiar to R developers, who expect pass-by-value semantics and side-effect-free functions (TOD q). Most R structures follow this paradigm, with the exception of index-assignment

(e.g. `x[2] <- 4.2`). Even in these cases, R's assign-by-value and copy-on-modify semantics ensure safety (`x <- y; x[2] <- 4.2` modifies `x` while preserving the original data in `y`).

Unfortunately, native R types do not emphasize performance – many operations require a full scan or copy of the data, making them prohibitively expensive for use in highly dynamic or large-data applications (TOD j). Attempts have been made to fill these gaps; we previously published an implementation of efficient, persistent stack and queue structures based on lazy evaluation techniques (noted as the only use of lazy structures in a 2019 review of the R ecosystem (TOD a)). In 2018 T. Barry surveyed the state of collection packages, identifying 11 packages providing set, map, stack, queue, and/or sequence structures (TOD b). Of these, only three implemented side-effect-free structures: **rstackdeque** (TOD u), **sets** (TOD w), and **S4Vectors** (TOD v). The latter two are optimized for bulk operations as compared to base-R types but, like R lists and vectors, do not support fast incremental insert/remove operations. Notably, **S4Vectors** is designed for extensibility and is widely used in the Bioconductor ecosystem, including support for interval ranges via **IRanges** (TOD g). Barry notes a need for fast, flexible, and side-effect-free data structures to support advanced use cases in the R ecosystem (TOD b).

Here we describe **immutables**, which implements a remarkable data structure known as monoid-annotated 2-3 finger trees. First described by Hinze and Paterson 20 years ago (Hinze and Paterson 2006), these provide persistent, amortized constant-time access to ends, $O(\log n)$ search/insertion/removal, and via monoid annotation can be purposed as other specialized structures (TOD d). We implement four such specialized structures: `flexseq`s, serving as indexable, splittable, and concatenable lists with end insertion/removal for stack and queue operations; priority queues, associating data elements with priority values for min/max access; ordered sequences, keeping items in order by provided keys; and interval indices, storing items with orderable ranges for overlap-like queries. Core operations are implemented in C++ via **Rcpp** (TOD s) for speed, and a developer API is provided to extend functionality via monoid annotation.

2. Overview

2.1. Flexible sequences

At the core of **immutables** is the `flexseq` structure, which operates similarly to R lists: they can store elements of any or mixed types, be named, and be indexed with the familiar `[`, `[[`, and `$` operators, including replacement. Elements can be inserted, accessed, or removed from either end via peek/pop/push operations in amortized constant time for efficient double-ended queue behavior. Additionally, `flexseq`s allow concatenation, splitting by position, and insertions and removals by position in $O(\log n)$ time.

```
x <- as_flexseq(letters[1:10])
x2 <- push_back(x, "z")
length(x)                  # 10
length(x2)                 # 11
x3 <- insert_at(x2, list("p", "q"))
length(x3)                 # 13
```

```
x4 <- x[1:5]              # flexseq of length 5
result <- pop_back(x3)
result$element            # "z"
result$remaining          # flexseq of length 12
```

Other implemented methods include `c()`, `as.list()`, `unlist()`, `fapply()` (analogous to `lapply()` for lists), and `print()`, mimicking `print.list()` but displaying only the first and last few elements.

While `flexseq`s support names, these are somewhat limited compared to named R lists: if names are used, every element must have a unique non-`NULL` name. Further, the usage of names degrades worst-case performance for some operations to a full $O(n)$ scan (for example, when concatenating), though front/back access remains $O(1)$ amortized in named contexts. Otherwise, all operations are $O(\log n)$ worst case, or $O(k \log n)$ when deleting k elements. For performance-sensitive applications requiring named access, ordered sequences (discussed below) can be associated with character-based names as ordering keys for fast access.

2.2. Priority queues

The priority-queue type stores arbitrary or mixed types, much like `flexseq`, but each element is associated with a priority value. Rather than efficient front/back access, priority queues provide efficient ($O(\log n)$) insertion and pop/peek by minimum and maximum priority value. Priorities may be any orderable type (such as dates), but full C++ optimization is available for primitive numeric, character, and logical priorities. Because multiple elements may share the same priority, "first" versus "all" access methods are available: `pop_all_min(q)` returns an object with two priority queues, `$elements` containing all elements with minimum priority and `$remaining` with the rest (`$elements` may be easily converted to base R via `as.list()`). Conversely, `pop_min(q)` returns an object with `$element`, the first (by insertion order) minimum-priority element, and `$remaining`, the rest of the priority queue.

```
q <- as_priority_queue(letters[1:6], priorities = c(2, 1, 3, 1, 9, 7))
peek_min(q)               # "b"
peek_all_min(q)           # priority queue w/ "b" "d"
q2 <- insert(q, "z", priority = 10)
res <- pop_max(q2)
res$element               # "z"
res$remaining             # priority queue w/ 6 entries
```

Internally, elements are stored in insertion order, though this should not be relied on. Accordingly, priority queues only support indexing with `[`, `[[`, and `$` by name, and the same caveats apply as for named `flexseq`s. Priority queues and other advanced types (below) may be cast down with `as_flexseq()` and `as.list()`, optionally preserving priority-value metadata.

2.3. Ordered sequences

The ordered-sequence type associates each element with a key value and keeps elements in key-value order. Keys may be any orderable type, with optimizations for primitive types, and

duplicate keys are kept in insertion order. Ordered sequences support efficient insert, peek, and pop by key, with "all" versions of peek and pop for edge-case duplicate-key use cases. Ordered sequences also support key counts and key-range queries.

```
o <- as_ordered_sequence(letters[1:6], keys = c(20, 10, 30, 10, 90, 70))
peek_key(o, 10)              # "b"
peek_all_key(o, 10)          # ordered sequence w/ "b" "d"
count_key(o, 10)             # 2
elements_between(o, 15, 85) # ordered seq w/ "a" "c" "f"
o2 <- insert(o, "z", key = 100)
res <- pop_key(o2, 100)
res$element                  # "z"
res$remaining                # ordered seq w/ "a" - "f"
```

As with other structures, ordered sequences may be named (the same name caveats apply) and indexed with `[`, `[[`, and `$`. Integer indices may be used with `[` and `[[`, in which case they refer to elements in key-sorted order. When `[` is used to subset an ordered sequence, the return is another ordered sequence and elements are kept in their original key-order. A warning is produced if this is not the requested order (e.g. `o[c(2,1)]`).

2.4. Interval indices

TODO — to be drafted by hand.

3. Benchmarks

TODO — to be drafted by hand.

4. Extension example

TODO — to be drafted by hand.

5. Conclusion

TODO — to be drafted by hand.

Computational details

TODO — to be drafted by hand.

Acknowledgments

References

(????a). "TODO: 2019 review of R ecosystem noting lazy structures." Placeholder.

(????b). "TODO: Barry (2018) survey of R collection packages." Placeholder.

(????c). "TODO: citation for deque uses (state search, scheduling)." Placeholder.

(????d). "TODO: citation for finger-tree specializations." Placeholder.

(????e). "TODO: citation for graph algorithms in R." Placeholder.

(????f). "TODO: citation for interval trees in computational geometry." Placeholder.

(????g). "TODO: citation for IRanges package." Placeholder.

(????h). "TODO: citation for miscellaneous/broad R applications." Placeholder.

(????i). "TODO: citation for ordered-sequence / range-query uses." Placeholder.

(????j). "TODO: citation for performance limits of native R types." Placeholder.

(????k). "TODO: citation for pretty trees in bioinformatics." Placeholder.

(????l). "TODO: citation for priority queue uses (graph, simulation)." Placeholder.

(????m). "TODO: citation for R in computational geometry." Placeholder.

(????n). "TODO: citation for R in machine learning." Placeholder.

(????o). "TODO: citation for R in simulation." Placeholder.

(????p). "TODO: citation for R in web services." Placeholder.

(????q). "TODO: citation for R pass-by-value / side-effect-free semantics." Placeholder.

(????r). "TODO: citation for R user interfaces (e.g. Shiny)." Placeholder.

(????s). "TODO: citation for Rcpp." Placeholder.

(????t). "TODO: citation for rpigpio / mobile computing in R." Placeholder.

(????u). "TODO: citation for rstackdeque package." Placeholder.

(????v). "TODO: citation for S4Vectors package." Placeholder.

(????w). "TODO: citation for sets package." Placeholder.

(????x). "TODO: Okasaki, Purely Functional Data Structures." Placeholder.

Hinze R, Paterson R (2006). "Finger Trees: A Simple General-Purpose Data Structure." *Journal of Functional Programming*, **16**(2), 197–217. doi:10.1017/S0956796805005769.

Affiliation:

Shawn T. O'Neil
TISLab
Translational and Integrative Sciences Lab
E-mail: shawn@tislab.org